



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Lifelong Adaptation in Minimal Structures: A Spiking-NEAT Architecture with R-STDP

Estudiante: Naiara Manteiga Cendón

Dirección: Outro Nome Completo

A Coruña, September 2026.

Dedicatoria

I would like to thank ...

Acknowledgements

I would like to thank ...

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Resumo

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Keywords:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Palabras chave:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.2.1	Main objective	2
1.2.2	Specific Objectives	3
1.3	Chosen approach	3
2	State of the Art	5
2.1	Neuroevolution and Genetic Algorithms	5
2.2	Spiking Neural Networks (SNN) and Green AI	10
2.3	Synaptic Plasticity and Biological Reinforcement Learning	12
2.4	Comparative Analysis and Gap Identification	13
3	Conclusións	15
	Bibliography	17

List of Figures

2.1	Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks.	6
2.2	The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings. Adapted from [1].	7
2.3	Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there is a preset chance that an inherited gene is disabled if it is disabled in either parent. Adapted from [1].	8

List of Tables

Introduction

1.1 Motivation

Artificial Neural Networks (ANNs) have been at the forefront of computer science research for the past few decades, the core concept of building artificial systems that simulate the biological behavior of the human brain remains a prominent research focus. In recent years, the capabilities of Deep Learning and transformer architectures have grown substantially, driven by their remarkable results and the commercialization of General Purpose Artificial Intelligence (GPAI). Nonetheless, these models suffer from two major shortcomings: their unsustainable energy and environmental consumption, and the massive initial training datasets they require. This severe energetic cost has given rise to initiatives such as Green AI [5], which advocates for computational efficiency as a primary evaluation criterion alongside model accuracy. Conversely, neuroevolutionary algorithms attempt to replicate both the mechanics of the brain and the evolutionary processes of living organisms. In particular, the NeuroEvolution of Augmented Topologies (NEAT) algorithm [1] offers a paradigm shift from its predecessors, yielding excellent results for continuous, high-dimensional control tasks, as will be detailed in subsequent sections.

The practical application of this project is heavily inspired by a well-known demonstration of the NEAT algorithm navigating a Super Mario Bros. environment, an implementation its creator named MarI/O [6]. This demonstration clearly illustrates that NEAT does not require massive training datasets nor any prior information about the environment. MarI/O begins the simulation without knowing the controls, discovers movement through trial and error, and evolves until it can clear the level autonomously. This effectively solves the first major shortcoming regarding initial data dependency.

However, standard NEAT does not resolve the energy consumption problem on its own. To address this, this project aims to implement two biologically inspired principles: Spiking Neural Networks (SNNs) [2] and Reward-Modulated Spike-Timing-Dependent Plasticity (R-

STDP) [3]. SNNs are built upon the premise that biological brains communicate via discrete energy "spikes". To put this into perspective, standard ANNs operate like an open tap where water (energy) flows continuously and without interruption; in contrast, adapting SNNs is akin to closing the tap to a bare minimum, allowing water to accumulate and only fall as a single drop once it reaches a specific density. Furthermore, the integration of R-STDP significantly enhances the evolutionary process. While standard NEAT focuses purely on evolving the network's topology, R-STDP focuses on dynamically evolving the synaptic weights during gameplay. This dual mechanism grants the agent greater adaptability, theoretically allowing the "brain" to generalize to new levels or similar tasks without the need to restart the training process from scratch.

Ultimately, the exploration of SNNs and R-STDP in this project aligns perfectly with the cutting edge of neuromorphic engineering. As modern technology increasingly shifts toward edge computing, autonomous robotics, and Internet of Things (IoT) devices, the demand for AI systems capable of real-time adaptation with minimal power consumption has never been higher. Therefore, validating these biological learning mechanisms within a dynamic software environment like Super Mario Bros not only addresses an immediate academic gap, but also directly contributes to the foundation of next generation, hardware deployable AI.

1.2 Objectives

1.2.1 Main objective

The primary objective of this project is to develop an autonomous Artificial Intelligence program, commonly referred to as an 'agent', capable of navigating and completing levels within the Super Mario Bros environment. In this context, an agent is an independent computational entity that continuously observes its surroundings and makes decisions to achieve a specific goal. This decision-making process will be driven by Reinforcement Learning, a machine learning paradigm in which the agent learns optimal behaviors through trial, error, and the reception of environmental rewards.

To achieve this while addressing the energy inefficiencies of modern AI, the project will substitute the computationally expensive continuous neural networks traditionally used in these tasks with a biologically plausible Spiking Neural Network (SNN). Furthermore, the agent's learning mechanism will be governed by Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP), enabling it to dynamically adapt its synaptic connections in real-time.

Ultimately, the goal is to prove that an agent built upon these biologically inspired principles can achieve high adaptability and learning efficiency in a dynamic environment while drastically minimizing theoretical energy consumption.

1.2.2 Specific Objectives

To achieve the main objective, the project is divided into the following sub-goals:

- **Establish a Baseline:** Implement and train a standard NEAT algorithm to play Super Mario Bros to serve as a performance and architectural baseline.
- **Implement Spiking Mechanics:** Replace standard continuous activation functions (e.g., Sigmoid) with a discrete Leaky Integrate-and-Fire (LIF) [2] neuron model to simulate biological spiking behavior.
- **Integrate Reinforcement Learning:** Substitute random weight mutation with Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP) to allow the agent to learn dynamically from environmental rewards (dopamine signals).
- **Evaluate Efficiency:** Compare the performance and theoretical computational energy consumption of the Spiking-NEAT agent against the standard NEAT baseline.

1.3 Chosen approach

To achieve the expected results, the project methodology is conceptually divided into three metaphorical pillars: the architect, the engine, and the teacher.

The Architect (Topology): is the foundational structure of the agent’s brain is generated using NeuroEvolution of Augmented Topologies (NEAT). NEAT improves upon earlier Topology and Weight Evolving Artificial Neural Networks (TWEANNs) by utilizing historical markings (innovation numbers), speciation, and incremental growth from minimal structures. Because NEAT has been validated on complex control tasks like double pole balancing, it serves as the ideal ”architect” to dynamically grow the network topology for this project.

The Engine (Activation Function): Standard NEAT typically employs a continuous sigmoidal activation function bounded between $[0, 1]$. However, because these activation values are continuous floating-point fractions, the underlying hardware is forced to perform computationally expensive Multiply Accumulate (MAC) operations for every single synapse, at every single time step. Even if a node’s activation is microscopically close to zero, the system must still execute dense matrix arithmetic across the entire network to calculate the next frame. This persistent, unconditional computation is exactly what drives the massive energy consumption problem acknowledged in the previous section. To resolve this, the network will be transformed into a Spiking Neural Network (SNN) by implementing the Leaky Integrate-and-Fire (LIF) model [2]. The LIF model is a mathematical abstraction of biological neurons focusing entirely on the timing of electrical spikes via three mechanics: integrate (accumulating incoming electrical signals), leak (the natural decay of voltage over time), and

fire (emitting a binary spike when the voltage surpasses a threshold and resetting). Expanding on the previous water tap metaphor, the LIF model acts as a bucket with a small hole that fills up until the bucket is full and gets emptied (resets) after. The tap represents synaptic current (integrate), the water level represents membrane potential, the hole represents natural leakage (leak), and the overflow when the bucket fills up completely represents the spike (fire). This discrete, binary event-driven approach is highly efficient because neurons only perform calculations when a discrete spike occurs, essentially reducing continuous matrix multiplication to sparse, binary addition.

The Teacher (Learning Rule): In standard NEAT, synaptic weights are mutated randomly between generations, which can result in inefficient learning. To address this, the project introduces Reinforcement Spike-Timing-Dependent Plasticity (R-STDP) as the "teacher". R-STDP acts as a biologically plausible reinforcement learning mechanism. Rather than randomly guessing weight values, R-STDP analyzes which neurons spiked immediately prior to a successful in-game action and strengthens those specific connections. By mimicking the release of dopamine in a biological brain, the model is directly encouraged to repeat productive behaviors.

Chapter 2

State of the Art

This chapter establishes the theoretical and mathematical foundations required to contextualize the contributions of this document. It provides a critical review of the current literature surrounding neuroevolutionary algorithms, Spiking Neural Networks (SNNs), and biologically plausible learning rules. By analyzing recent advancements and their inherent limitations, this section constructs the conceptual framework necessary to justify the proposed Spiking-NEAT architecture and its application in dynamic control environments.

2.1 Neuroevolution and Genetic Algorithms

Historically, the standard method for training Artificial Neural Networks (ANNs) is the backpropagation algorithm[cite: 8]. The fundamental purpose of backpropagation is to optimize a network's predictions through supervised learning by systematically adjusting its internal weights and biases to minimize a mathematical error[cite: 8].

As illustrated in Figure 2.1, this training process operates in three distinct phases[cite: 8]:

1. **The Forward Pass:** Input data propagates forward through the network's interconnected hidden layers[cite: 8]. Each node extracts features and applies continuous mathematical transformations until the final layer outputs a prediction[cite: 8].
2. **Error Calculation:** A predefined loss function evaluates this output, measuring the mathematical discrepancy (the error) between the network's prediction and the actual "ground truth" provided by the training dataset[cite: 8].
3. **The Backward Pass:** The algorithm calculates the gradient of the loss function[cite: 8]. It then moves backward through the network, effectively distributing the "blame" for the error by calculating exactly how much each individual weight and bias needs to be adjusted to improve the next prediction[cite: 8].

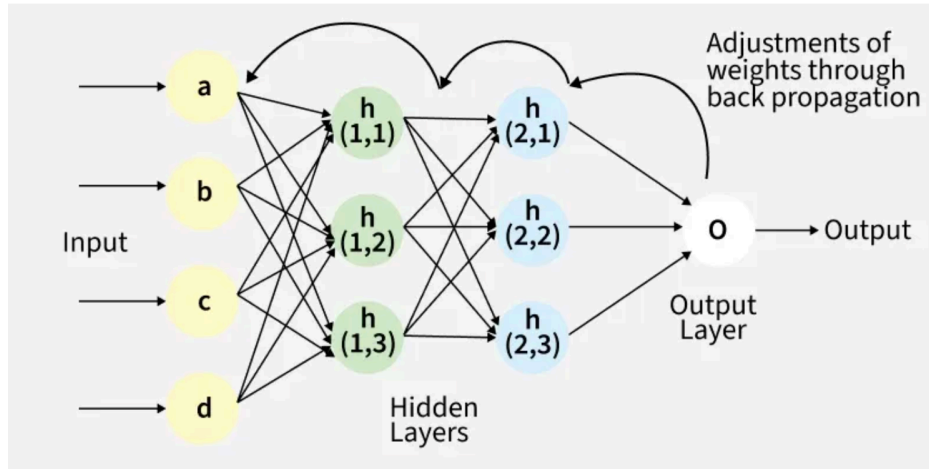


Figure 2.1: Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks.

While this gradient descent method is incredibly powerful for supervised tasks like image classification, it possesses two strict requirements: it relies on massive datasets of “correct answers” to calculate the initial loss, and it requires continuous, differentiable activation functions to calculate the gradients[cite: 8].

However, in dynamic control environments (such as autonomous robotics or video games like Super Mario Bros) there is no dataset of “perfect moves” to learn from[cite: 8]. The AI only knows whether it survived or failed based on sparse environmental rewards[cite: 8]. Because standard gradient-based backpropagation struggles in these unpredictable, data-poor scenarios, researchers turned to biology for a solution, giving rise to Neuroevolution (NE)[cite: 8].

Neuroevolution is a prominent subfield of Artificial Intelligence that applies the principles of Darwinian natural selection to neural networks. Instead of training a single network with a dataset, Neuroevolution relies on Genetic Algorithms (GAs) to generate a “population” of many different candidate networks. The algorithm evaluates each network based on its performance in the environment (its fitness). The weakest networks are discarded, while the most successful ones are selected to produce a new generation through biological mechanics like crossover (combining traits from two parents) and mutation (random structural changes).

TWEANNs

Historically, early neuroevolutionary systems focused exclusively on optimizing the synaptic weights of a neural network, while its structural topology remained fixed by a human engineer. However, guessing the optimal architecture for a complex, dynamic environment is

incredibly difficult. To overcome this, researchers developed Topology and Weight Evolving Artificial Neural Networks (TWEANNs) [1], algorithms designed to evolve both the connection weights and the physical structure of the network simultaneously.

While TWEANNs represented a significant conceptual leap, their practical application was severely hindered by three critical limitations:

1. **The Competing Conventions (Permutation) Problem:** This is the most severe flaw in early neuroevolution. It occurs when two parent networks learn to solve the same problem, but encode their solutions using different internal topological orderings. If a genetic algorithm attempts to cross over (breed) these two parents, the resulting offspring will often suffer from missing or duplicated features. Because the algorithm has no way to align the mismatched hidden nodes, crossing over two highly successful parents frequently produces damaged offspring.

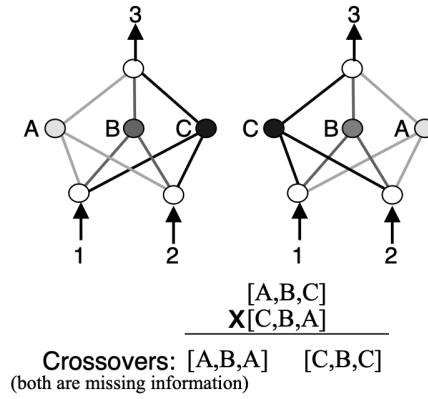


Figure 1: The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings.

Figure 2.2: The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings. Adapted from [1].

As illustrated in Figure 2.2, crossing over two functional networks with different internal orderings results in offspring missing crucial information (e.g., missing node C or node A).

2. **Unprotected Innovation:** In evolutionary algorithms, a newly mutated structure (like

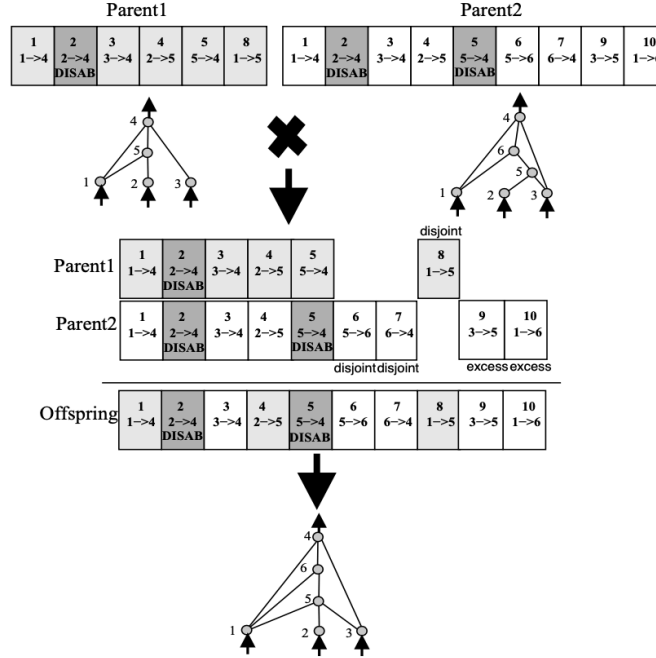


Figure 4: Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there's a preset chance that an inherited gene is disabled if it is disabled in either parent.

Figure 2.3: Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there is a preset chance that an inherited gene is disabled if it is disabled in either parent. Adapted from [1].

a new hidden node) often causes a temporary drop in performance before its weights can be optimized. Early TWEANNs evaluated all networks in a single, unified pool. Consequently, new and complex topological innovations were immediately driven to extinction by older, structurally simpler networks that had already optimized their weights. Also, having the global population all together can cause the initially “stronger” genes to prevail and drive other species of the global population to extinction; this is a hazard for innovation.

3. **Topological Bloat:** To ensure genetic diversity, early models frequently initialized populations with complex, highly randomized topologies. This bloated the genomes with unjustified connections from the very first generation, unnecessarily expanding the mathematical search space and wasting computational resources.

These three structural flaws meant that TWEANNs struggled to scale to high-dimensional tasks. The field required a paradigm shift to make neuroevolution mathematically efficient and viable for complex control environments, a shift that was realized by the introduction of the NEAT algorithm.

NeuroEvolution of Augmented Topologies (NEAT)

NeuroEvolution of Augmented Topologies (NEAT) is the most efficient solution for the problems that TWEANNs presented in the past, giving a practical solution for each problem:

“We claim that the increased efficiency is due to (1) employing a principled method of crossover of different topologies, (2) protecting structural innovation using speciation, and (3) incrementally growing from minimal structure.” [1]

1. **Historical Markings and Crossover:** NEAT is based on the premise that genomes can have different sizes and structures (mimicking gene amplification in biology), which makes crossover a hard task due to variable-length genomes. This is why direct genetic encoding and innovation numbers are used to approximate the homology of two genes. Basically, crossover happens when two genes have compatible historical markings (innovation numbers, which are IDs for the markings). Using artificial synapsis allows the genes to add new structures without losing track of which gene is which; this keeps crossover orderly and allows the right genes to mate, solving the Competing Conventions problem.
2. **Speciation:** NEAT uses speciation (or niching) to divide the population into distinct species based on topological similarity. This is necessary since having the global population all together can cause the initially “stronger” genes to prevail and drive other

species of the global population to extinction, which is a hazard for innovation. Using explicit fitness sharing, speciating is possible since it forces individuals with similar genomes to share their fitness payoff (using fitness peaks), protecting initially “weaker” species from becoming extinct and protecting innovation in their own species.

3. **Minimal Initial Structures:** The challenge observed in TWEANNs with the initial population is the initial topological complexity. Early TWEANNs started with random complex topologies to ensure diversity, which bloated the genomes with unjustified connections and expanded the search space. NEAT solves this by starting with a completely minimal population (no hidden nodes) and letting complexity emerge incrementally only when it provides a functional benefit. This minimizes the dimensionality of the search space, drastically boosting performance.

In general, using these three key points, NEAT not only solves the initial problems of TWEANNs but also serves as a great contribution for GAs:

“NEAT is also an important contribution to GAs because it shows how it is possible for evolution to both optimize and complexify solutions simultaneously, offering the possibility of evolving increasingly complex solutions over generations, and strengthening the analogy with biological evolution.” [1]

Moreover, as will be discussed in the following sections, it has also passed thorough testing and ablations.

Innovation Numbers ...

2.2 Spiking Neural Networks (SNN) and Green AI

As previously mentioned, the world is currently experiencing the rapid rise and commercialization of AI and GPAI. However, there is ongoing dispute regarding the computational cost of running these machines and the vast volumes of data they require. Most of these modern architectures fall under the category of Red AI. According to Schwartz et al., “Red AI refers to AI research that seeks to obtain state-of-the-art results in accuracy (or related measures) through the use of massive computational power—essentially ‘buying’ stronger results” [5]. While Red AI focuses primarily on maximizing accuracy through sheer computational force, Green AI advocates for treating energy efficiency and computational cost as primary evaluation metrics alongside accuracy [5]. Consequently, this project evaluates energy efficiency as a core metric, operating under the premise that it is just as valuable as model accuracy.

Furthermore, the primary reason these standard implementations are so computationally (and therefore energetically) expensive is the activation function. The activation function is a mathematical equation that determines whether a given node should remain inactive or fire, defining its output. The vast majority of Red AI alternatives utilize continuous activation functions, such as Sigmoid or ReLU. The fundamental issue is that these functions rely on continuous floating-point numbers, requiring the underlying hardware to perform complex matrix floating-point operations constantly.

As Schwartz et al. observe: “Ironically, deep learning was inspired by the human brain, which is remarkably energy efficient” [5]. As mentioned previously, a biological brain does not physically operate like an open tap, rather, it processes information via discrete spikes. A biological neuron is divided into three main components: dendrites, the soma, and the axon. Roughly speaking, the dendrites can be considered the “input system,” the soma acts as the central processor or “GPU,” and the axon serves as the “output system” [2]. Electrical energy travels through these components, and a sufficient accumulation of voltage triggers an action potential. A group of these action potentials over a given time period constitutes a spike. These biological mechanics form the foundational principles of Spiking Neural Networks (SNNs) [2]. Utilizing spikes as an activation mechanism renders the architecture significantly more energy-efficient, as it relies on discrete, binary event-driven signals rather than continuous floating-point operations. Furthermore, SNNs are fundamentally inspired by electrical circuitry, making them highly optimized for neuromorphic hardware translation.

However, for an SNN to function computationally, it requires a physical mathematical abstraction. In this project, that abstraction is the Leaky Integrate-and-Fire (LIF) model [2]. As the name suggests, the LIF model governs neuronal dynamics through three distinct stages: integrate, leak, and fire. First, the neuron accumulates energy from incoming synaptic spikes, which raises its internal membrane potential (integration). Second, the membrane possesses a natural electrical resistance, if spikes do not arrive fast enough, the membrane progressively loses voltage, returning toward its resting state (leak). Finally, if the integration outpaces the leak and surpasses a strict voltage threshold, the neuron instantly emits a binary spike and resets its internal voltage to a baseline (fire). In this architecture, every continuous node generated by the NEAT algorithm is replaced by this discrete LIF physical model.

This section has outlined the significant computational advantages of SNNs and the LIF model, demonstrating why they serve as a powerful framework for achieving Green AI efficiency. While combining the NEAT algorithm with LIF neurons yields an exceptionally efficient Spiking-NEAT architecture, it introduces a critical challenge: this discrete, non-differentiable implementation cannot be trained using standard continuous mathematics, such as Backpropagation. A Spiking-NEAT architecture requires a biologically plausible learning

rule to optimize its weights, which leads to the final core component of this project: biological Reinforcement Learning.

2.3 Synaptic Plasticity and Biological Reinforcement Learning

Most of the conventional AI architectures discussed previously rely on continuous activation functions and smooth, differentiable curves, therefore, the standard method for training these Artificial Neural Networks (ANNs) is Backpropagation. Because Backpropagation calculates gradients using loss and error functions, it strictly requires continuous values. In contrast, the Spiking-NEAT approach utilizes non-differentiable, discrete binary signals (spikes). Consequently, traditional gradient-descent methods cannot be applied. Furthermore, relying solely on NEAT's random weight mutations between evolutionary generations is computationally slow and inefficient. Therefore, this project proposes an alternative, biologically plausible solution to dynamically evolve the synaptic weights while the agent interacts with the environment.

In biological brains, learning is driven by synaptic plasticity, which refers to the ability of neural connections to strengthen or weaken over time based on their activity. This correlates with the Hebbian postulate: "Neurons that fire together, wire together." This rule states that if neuron A repeatedly and persistently takes part in firing neuron B, the synaptic efficacy from A to B increases [2]. These principles are mathematically formalized in Spike-Timing-Dependent Plasticity (STDP). In STDP, the change in synaptic weight depends heavily on the precise timing of spikes. Generally, if a pre-synaptic neuron fires just before a post-synaptic neuron, the connection between them is strengthened, a process known as Long-Term Potentiation (LTP). Conversely, if the pre-synaptic neuron fires just after the post-synaptic neuron, the connection is weakened, resulting in Long-Term Depression (LTD).

However, standard STDP is strictly unsupervised, it evaluates only the relative timing of spikes, lacking any mechanism to determine whether the resulting action was beneficial or detrimental to the system. Consequently, if a negative action occurs repeatedly, STDP will continually strengthen the synaptic pathways responsible for that action simply because the neurons fire in sequence. Without a goal-oriented mechanism, standard STDP can reinforce maladaptive behaviors, ultimately hindering the agent's progression in dynamic environments.

To resolve this credit assignment problem, Reward-Modulated STDP (R-STDP) is introduced [4]. R-STDP integrates the principles of Reinforcement Learning by expanding standard STDP into a three factor learning rule. The three key factors are: the pre-synaptic spike, the post-synaptic spike, and a delayed neuromodulatory signal, which mathematically mimics the release of dopamine in biological brains. In this model, when neurons fire sequentially,

STDP initially marks the connection with a temporary “eligibility trace.” If the agent subsequently performs a beneficial action and receives an environmental reward (a dopamine boost) shortly after, this trace is converted into a permanent weight increase (LTP). Conversely, if the agent receives a punishment or no reward, the connection is actively weakened (LTD) [4]. Consequently, R-STDP not only evolves the network weights dynamically but also ensures that learning is goal-directed and efficient.

In conclusion, this triad forms the complete theoretical foundation of the proposed model: NEAT acts as the architect to build the minimal network topology, LIF neurons serve as the highly energy-efficient processing engine using discrete spikes, and R-STDP functions as the biological reinforcement learning mechanism, adjusting synaptic weights dynamically based on behavioral success rather than isolated timing.

2.4 Comparative Analysis and Gap Identification

In summary, the theoretical frameworks discussed thus far establish several key premises: NEAT is a highly effective neuroevolutionary algorithm that dynamically evolves topologies but incurs high computational and energetic costs. Spiking Neural Networks, specifically utilizing the LIF model, offer a paradigm shift toward energy efficiency by replacing continuous floating-point operations with discrete binary spikes; however, these non-differentiable networks cannot be trained via Backpropagation. Finally, R-STDP provides a biologically plausible reinforcement learning mechanism to evolve synaptic weights dynamically, overcoming the limitations of unsupervised STDP by distinguishing between beneficial and detrimental actions.

The individual viability of these mechanisms has been validated in recent literature. Qiu et al. [3] demonstrated that a Spiking-NEAT architecture is not only possible but can outperform standard NEAT in specific control tasks. Similarly, Mozafari et al. [4] provided substantial evidence that R-STDP is an effective learning rule for SNNs. However, despite these isolated successes, a distinct research gap remains: these methodologies have rarely been unified and evaluated together within a highly complex, dynamic environment. Therefore, this project aims to prove that the integration of a Spiking-NEAT architecture, LIF neuron models, and R-STDP learning is a viable and highly efficient solution for complex control tasks.

To benchmark this integration, the agent is tasked with navigating *Super Mario Bros.*, an environment directly inspired by the neuroevolution showcase MarI/O [6]. While utilizing a commercial video game may intuitively seem informal, such environments have become the gold standard for benchmarking modern reinforcement learning models. Unlike simple balancing tasks, *Super Mario Bros.* is a highly dynamic, non-linear control environment characterized by a high-dimensional observation space. Because the agent must interpret

the game state from raw visual pixel data, the computational burden of processing spatial relationships, momentum-based physics, and moving obstacles is exceptionally high. This complexity makes it an ideal testing ground to validate whether an energy-efficient Spiking Neural Network can match the performance of computationally expensive standard neural architectures in complex, real-time tasks. Furthermore, successfully solving this environment serves as a proof of concept for broader applications in robotics and aerospace, where extreme energy constraints and high-dimensional data processing are critical.

Ultimately, the objective of this thesis is to provide a thorough comparative analysis, demonstrating how a Spiking-NEAT architecture guided by R-STDP can achieve comparable or superior results to a standard NEAT baseline, while significantly reducing overall energy consumption and accelerating convergence.

Conclusión

Bibliography

- [1] K. O. Stanley and R. Miikkulainen, “Evolving neural networks through augmenting topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.
- [3] H. Qiu, M. Garratt, D. Howard, and S. Anavatti, “Evolving spiking neural networks for nonlinear control problems,” in *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2018, pp. 1367–1373.
- [4] M. Mozafari, S. R. Kheradpisheh, T. Masquelier, A. Nowzari-Dalini, and M. Ganjtabesh, “First-spike-based visual categorization using reward-modulated STDP,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 12, pp. 6178–6190, 2018.
- [5] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [6] SethBling, “MarI/O - machine learning for video games,” <https://www.youtube.com/watch?v=qv6UVOQ0F44>, 2015, accessed: 2026-09-08.
- [7] TechWithTim, “NEAT-flappy-bird: A neuroevolution implementation of the classic flappy bird game using the NEAT algorithm,” <https://github.com/techwithtim/NEAT-Flappy-Bird>, 2019, gitHub Repository. Accessed: 2026-09-08.